

PRAKTIKUM SISTEM OPERASI

**MODUL XI
(MEMORI XINU)**



Oleh:

(Naura Aisha Zahira)

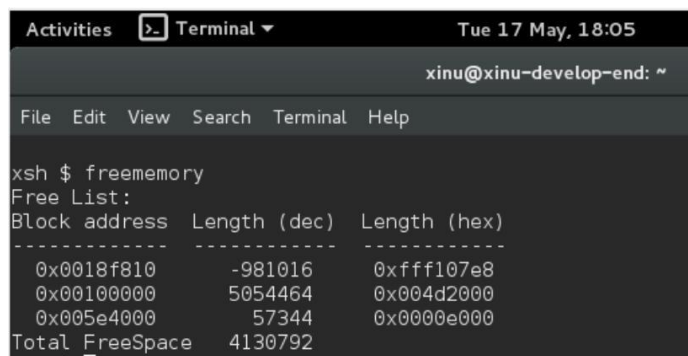
(2311104078)

**PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026**

JURNAL

1. Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:
 - a. Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Output yang diinginkan:

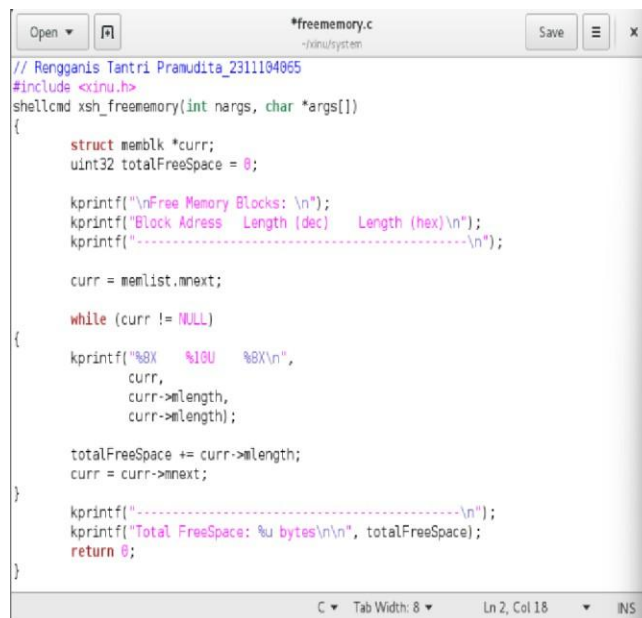


```
Activities Terminal Tue 17 May, 18:05
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help

xsh $ freememory
Free List:
Block address Length (dec) Length (hex)
-----
0x0018f810 -981016 0xffff107e8
0x00100000 5054464 0x004d2000
0x005e4000 57344 0x0000e000
Total FreeSpace 4130792
```

Jawab:

- syntax xinu/system/freememory.c



```
*freememory.c
-xinu/system

// Rengganis Tantri Pramudita_2311104065
#include <xinu.h>
shellcmd xsh_freememory(int nargs, char *args[])
{
    struct memblk *curr;
    uint32 totalFreeSpace = 0;

    kprintf("\nFree Memory Blocks: \n");
    kprintf("Block Address Length (dec) Length (hex)\n");
    kprintf("-----\n");

    curr = memlist.mnext;

    while (curr != NULL)
    {
        kprintf("%0X %10U %0X\n",
            curr,
            curr->length,
            curr->length);

        totalFreeSpace += curr->length;
        curr = curr->mnext;
    }

    kprintf("-----\n");
    kprintf("Total FreeSpace: %u bytes\n\n", totalFreeSpace);
    return 0;
}
```

- Outputnya

```
Welcome to Xinu!

xsh $ freememory

Free Memory Blocks:
Block Address  Length (dec)  Length (hex)
-----
18E110        U   18E110    FFF11EE8
100000        U   100000    4D8000
5E8000        U   5E8000     C000
-----
Total FreeSpace: 4153064 bytes
```

2. Jawablah pertanyaan berikut:

- a. Mengapa Xinu memisahkan data segment dan BSS segment?

Jawab:

Xinu memisahkan data segment dan BSS segment agar pengelolaan memori lebih efisien, di mana data segment menyimpan variabel global yang sudah diinisialisasi, sedangkan BSS segment menyimpan variabel global yang belum diinisialisasi sehingga tidak perlu memakan ruang di file program.

- b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Jawab:

Selama eksekusi, alokasi memori akan mengurangi free space karena digunakan oleh proses, sedangkan dealokasi akan menambah kembali free space, namun proses ini bisa menyebabkan fragmentasi sehingga penggunaan memori menjadi kurang optimal.

- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Jawab:

Heap lebih berpotensi menimbulkan masalah dibandingkan stack karena pengelolaannya dilakukan secara manual, dapat menyebabkan fragmentasi dan memory leak, sedangkan stack bersifat otomatis dan terstruktur sehingga lebih aman.

- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Jawab:

Xinu menggunakan struktur linked list untuk menyimpan free block karena ukuran memori yang tersedia bersifat dinamis, sehingga linked list memudahkan proses penambahan, penghapusan, dan pengelolaan blok memori secara fleksibel.

- e. Apa tantangan utama dalam penggunaan heap di Xinu?

Jawab:

Tantangan utama dalam penggunaan heap di Xinu adalah adanya fragmentasi memori, kemungkinan terjadinya memory leak, serta risiko kehabisan memori akibat alokasi yang tidak terkontrol.